Playwright Test Automation Report

End-to-End Test Automation – SauceDemo E-Commerce Platform

Prepared by: Rahaf Mohammed Almohammadi

June 2026

1. Introduction

This report presents an automated end-to-end (E2E) testing project built for the SauceDemo e-commerce demo application using Playwright. The objective of the project is to validate the application's core user journeys – login, product browsing, cart management, and checkout – through a structured, maintainable automation framework, and to integrate that framework into a continuous integration (CI) pipeline so that regressions are detected automatically on every code change.

The project, named playwright-ecommerce-tests, is built in JavaScript using the Playwright test runner, and follows the Page Object Model (POM) design pattern to separate page interaction logic from test logic. Tests are organized into three categories – Smoke, End-to-End, and Regression – and execute across multiple browser engines and a mobile viewport, with automatic failure diagnostics (screenshots, video, and trace) and an automated CI/CD pipeline via GitHub Actions.

2. Step-by-Step Implementation Walkthrough

This section documents the exact sequence followed to build the project, from the very first configuration file to the final test run. Each step explains what was implemented, why it was implemented that way, and includes the complete corresponding source code, so the entire project can be reproduced and understood end to end.

Step 1: Initialize the Project

The project starts as a plain Node.js project. Run the following commands inside an empty folder named playwright-ecommerce-tests to initialize it and install Playwright as a dev dependency:

```
npm init -y
npm install -D @playwright/test
npx playwright install
```

This creates a package.json file and downloads the Chromium, Firefox, and WebKit browser binaries that Playwright needs to run tests.

Step 2: Configure Playwright (playwright.config.js)

This file controls how every test in the project behaves: which folder to scan for tests, how many times to retry on failure, which browsers to run against, and what diagnostic artifacts to capture.

```
// @ts-check
const { defineConfig, devices } = require('@playwright/test');

module.exports = defineConfig({
  testDir: './tests',
  timeout: 60000,
```

```
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,

  reporter: [
    ['html', { outputFolder: 'playwright-report', open: 'never' }],
    ['list'],
    ['json', { outputFile: 'test-results/results.json' }],
  ],

  use: {
    baseURL: 'https://www.saucedemo.com',
    screenshot: 'only-on-failure',
    video: 'on-first-retry',
    trace: 'on-first-retry',
    headless: false,
    viewport: { width: 1280, height: 720 },
  },

  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: {
        ...devices['Desktop Firefox'],
        actionTimeout: 15000,
        navigationTimeout: 60000,
      },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    {
      name: 'mobile-chrome',
      use: { ...devices['Pixel 5'] },
    },
  ],

  outputDir: 'test-results/',
});
```

- testDir: './tests' – tells Playwright where to look for test files
- fullyParallel: true – runs tests concurrently to save time
- retries / workers – retries failed tests twice and limits parallel workers to 2, but only when running on CI (process.env.CI)
- reporter – generates three report formats simultaneously: HTML, a console list, and a raw JSON file

- use – shared settings for every test: the application's baseURL, screenshots on failure only, video and trace capture on the first retry, headed mode, and a fixed 1280×720 viewport
- projects – defines four execution environments: Chromium, Firefox (with extended timeouts), WebKit, and a mobile-chrome profile emulating a Pixel 5

Step 3: Define npm Scripts (package.json)

With the configuration in place, package.json was updated with convenience scripts so different parts of the suite can be triggered with short commands instead of typing full Playwright CLI commands every time.

```
{
  "name": "playwright-ecommerce-tests",
  "version": "1.0.0",
  "description": "E2E Automation Testing Suite for SauceDemo using Playwright",
  "scripts": {
    "test": "npx playwright test",
    "test:smoke": "npx playwright test tests/smoke --project=chromium",
    "test:e2e": "npx playwright test tests/e2e --project=chromium",
    "test:regression": "npx playwright test tests/regression",
    "test:all-browsers": "npx playwright test",
    "test:headed": "npx playwright test --headed",
    "report": "npx playwright show-report",
    "codegen": "npx playwright codegen https://www.saucedemo.com"
  },
  "devDependencies": {
    "@playwright/test": "^1.58.2",
    "@types/node": "^25.5.0"
  }
}
```

- test / test:all-browsers – run the entire suite on all configured browsers
- test:smoke, test:e2e, test:regression – run only a specific test folder, which is useful for quick, targeted checks during development
- report – opens the last HTML report
- codegen – launches Playwright's interactive recorder against the SauceDemo site to generate locators and actions automatically

Step 4: Set Up the CI/CD Pipeline (.github/workflows/playwright.yml)

Before writing any actual tests, the CI pipeline was set up so that, from this point forward, every single push or pull request would automatically install the project, install the browsers, and run whatever tests existed – catching integration issues as early as possible.

```
name: Playwright Tests
on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]
jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
```

```
    - uses: actions/setup-node@v4
      with:
        node-version: lts/*
    - name: Install dependencies
      run: npm ci
    - name: Install Playwright Browsers
      run: npx playwright install --with-deps
    - name: Run Playwright tests
      run: npx playwright test
    - uses: actions/upload-artifact@v4
      if: ${{ !cancelled() }}
      with:
        name: playwright-report
        path: playwright-report/
        retention-days: 30
```

- Triggers (on:) – runs on every push or pull request targeting the main or master branch
- checkout + setup-node – pulls the repository code and installs Node.js (LTS) on a fresh Ubuntu virtual machine
- npm ci – installs exact dependency versions from package-lock.json
- playwright install --with-deps – installs the browsers plus their required system libraries
- npx playwright test – runs the full suite
- upload-artifact – publishes the HTML report for 30 days, even if the run was not cancelled but some tests failed

Step 5: Create Test Data Fixtures (fixtures/users.js)

Before writing tests, the different SauceDemo user accounts were centralized into a single fixture file. This avoids hard-coding credentials inside every test file and makes it trivial to add new test scenarios later.

```
const users = {
  standard: {
    username: 'standard_user',
    password: 'secret_sauce'
  },
  locked: {
    username: 'locked_out_user',
    password: 'secret_sauce'
  },
  problem: {
    username: 'problem_user',
    password: 'secret_sauce'
  },
  performance: {
    username: 'performance_glitch_user',
    password: 'secret_sauce'
  },
  error: {
    username: 'error_user',
    password: 'secret_sauce'
  },
  visual: {
    username: 'visual_user',
    password: 'secret_sauce'
```

```
  },
  invalid: {
    username: 'wrong_user',
    password: 'wrong_pass'
  }
};
module.exports = users;
```

- standard / locked / invalid – the three user types actually exercised in the current test suite
- problem / performance / visual / error – additional SauceDemo accounts kept ready in the fixture for future test coverage (UI bugs, performance lag, and visual regressions, respectively)

Step 6: Build the Login Page Object (pages/LoginPage.js)

The first Page Object created was LoginPage, since every other test in the suite depends on logging in first. It stores the locators for the login screen and exposes simple actions on top of them.

```
class LoginPage {
  constructor(page) {
    this.page = page;

    // Locators
    this.usernameInput    = page.locator('#user-name');
    this.passwordInput    = page.locator('#password');
    this.loginButton      = page.locator('#login-button');
    this.errorMessage     = page.locator('[data-test="error"]');
    this.burgerMenu       = page.locator('#react-burger-menu-btn');
    this.logoutButton     = page.locator('#logout_sidebar_link');
  }

  async navigate() {
    await this.page.goto('/');
  }

  async login(username, password) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async getErrorMessage() {
    return await this.errorMessage.textContent();
  }

  async logout() {
    await this.burgerMenu.click();
    await this.logoutButton.click();}
}

module.exports =  LoginPage ;
```

- navigate() – opens the application's base URL
- login(username, password) – fills both fields and submits the form in one call

- getErrorMessage() – returns the text of any validation/login error, used later by both smoke and regression tests
- logout() – opens the side menu first (the logout link is hidden until the menu is open), then clicks logout

[Insert screenshot here – Login Page]

Step 7: Build the Products / Inventory Page Objects (pages/ProductsPage.js & pages/InventoryPage.js)

Immediately after login, the user lands on the product listing screen. Two Page Objects were created for it – ProductsPage (used by the cart/checkout flow) and InventoryPage (used by the smoke suite to confirm the page loaded after login) – both pointing at the same underlying screen.

```
class ProductsPage {
    constructor(page) {
        this.page = page;

        // Locators
        this.sortDropdown = page.locator('[data-test="product-sort-container"]');
        this.productNames = page.locator('[data-test="inventory-item-name"]');
        this.addToCartBtn = page.locator('[data-test="add-to-cart-sauce-labs-
backpack"]');
        this.cartIcon     = page.locator('.shopping_cart_link');
    }

    async sortBy(option) {
        await this.sortDropdown.selectOption(option);
    }

    async getFirstProductName() {
        return await this.productNames.first().textContent();
    }

    async addFirstItemToCart() {
        await this.addToCartBtn.click();
    }

    async goToCart() {
        await this.cartIcon.click();
    }
}

module.exports = ProductsPage;
```

```
class InventoryPage {
  constructor(page) {
    this.page = page;

    this.pageTitle      = page.locator('[data-test="title"]');
    this.inventoryItems = page.locator('.inventory_item');
    this.cartIcon       = page.locator('.shopping_cart_link');
    this.productNames   = page.locator('[data-test="inventory-item-name"]');
    this.addToCartBtn   = page.locator('[data-test="add-to-cart-sauce-labs-
backpack"]');
```

```
    this.sortDropdown   = page.locator('[data-test="product-sort-container"]');

  }

  async getTitle() {
    return await this.pageTitle.textContent();
  }

  async isLoaded() {
    return await this.inventoryItems.count() > 0;
  }

  async getProductCount() {
    return await this.inventoryItems.count();
  }


    async addFirstItemToCart() {
        await this.addToCartBtn.click();
    }

    async sortBy(option) {
        await this.sortDropdown.selectOption(option);
    }

    async getFirstProductName() {
        return await this.productNames.first().textContent();
    }
}

module.exports = InventoryPage;
```

- addFirstItemToCart() – adds the first listed product to the cart
- goToCart() – navigates to the cart screen
- getTitle() (InventoryPage) – reads the page heading, used to confirm a successful login redirected to “Products”

[Insert screenshot here – Products / Inventory Page]

Step 8: Build the Cart Page Object (pages/CartPage.js)

Next in the purchase flow is the cart screen, where the user reviews items before checking out.

```
class CartPage {
    constructor(page) {
        this.page = page;

        this.cartItems        = page.locator('[data-test="inventory-item"]');
        this.removeBtn        = page.locator('[data-test="remove-sauce-labs-
backpack"]');
        this.checkoutBtn      = page.locator('[data-test="checkout"]');
        this.continueShopBtn  = page.locator('[data-test="continue-shopping"]');
    }

    async getItemCount() {
        return await this.cartItems.count();
```

```
    }

    async removeItem() {
        await this.removeBtn.click();
    }

    async clickCheckout() {
        await this.checkoutBtn.click();
    }

    async continueShopping() {
        await this.continueShopBtn.click();
    }
}

module.exports = CartPage;
```

- getItemCount() – returns how many items are currently in the cart
- clickCheckout() – proceeds into the checkout flow, used by the E2E and regression tests

[Insert screenshot here – Cart Page]

Step 9: Build the Checkout Page Object (pages/CheckoutPage.js)

The final Page Object covers the two-step checkout form: entering customer information, then confirming the order.

```
class CheckoutPage {
    constructor(page) {
        this.page = page;

        // Checkout step 1 locators
        this.firstNameInput = page.locator('[data-test="firstName"]');
        this.lastNameInput  = page.locator('[data-test="lastName"]');
        this.zipCodeInput   = page.locator('[data-test="postalCode"]');
        this.continueBtn    = page.locator('[data-test="continue"]');
        this.cancelBtn      = page.locator('[data-test="cancel"]');
        this.errorMessage   = page.locator('[data-test="error"]');

        // Checkout step 2 locators
        this.finishBtn         = page.locator('[data-test="finish"]');
        this.confirmationMsg   = page.locator('[data-test="complete-header"]');
    }

    async fillInfo(firstName, lastName, zip) {
      await this.firstNameInput.fill(firstName);
      await this.lastNameInput.fill(lastName);
      await this.zipCodeInput.fill(zip);
    }

    async clickContinue() {
      await this.continueBtn.click();
    }

    async clickCancel() {
      await this.cancelBtn.click();
```

```
    }

    async getErrorMessage() {
      return await this.errorMessage.textContent();
    }

    async clickFinish() {
      await this.finishBtn.click();
    }

    async getConfirmation() {
      return await this.confirmationMsg.textContent();
    }
}

module.exports = CheckoutPage;
```

- fillInfo(firstName, lastName, zip) – fills the three required fields in one call
- clickContinue() / clickFinish() – advances through the two checkout steps
- getErrorMessage() – reads any field-validation error, used by the regression/negative test
- getConfirmation() – reads the final order-confirmation message, used by the E2E test

[Insert screenshot here – Checkout Page]

Step 10: Write the Smoke Test Suite (tests/smoke/login.spec.js)

With all the Page Objects and fixtures in place, the first test file written was the smoke suite, since validating login is the most fundamental check before testing anything deeper in the application. It contains seven test cases (TC-001 to TC-007) covering successful login, a locked-out user, invalid credentials, each field left empty individually and together, and logout.

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../../pages/LoginPage');
const InventoryPage = require('../../pages/InventoryPage');
const users = require('../../fixtures/users');

test.describe('Login Tests', () => {
    let loginPage;

    test.beforeEach(async ({ page }) => {
        loginPage = new LoginPage(page);
        await loginPage.navigate();
    });

    // TC-001
    test('should login successfully with valid credentials', async ({ page }) => {
        await loginPage.login(users.standard.username, users.standard.password);
        const inventoryPage = new InventoryPage(page);
        const title = await inventoryPage.getTitle();
        expect(title).toBe('Products');
    });

    // TC-002
    test('should show error for locked out user', async ({ page }) => {
        await loginPage.login(users.locked.username, users.locked.password);
```

```
        const error = await loginPage.getErrorMessage();
        expect(error).toContain('Sorry, this user has been locked out');
    });

    // TC-003
    test('should show invalid credentials', async ({ page }) => {
        await loginPage.login(users.invalid.username, users.invalid.password);
        const error = await loginPage.getErrorMessage();
        expect(error).toContain('Username and password do not match any user in this
service');
    });

    // TC-004
    test('should show error when username is empty', async ({ page }) => {
        await loginPage.login('', users.standard.password);
        const error = await loginPage.getErrorMessage();
        expect(error).toContain('Username is required');
    });

    // TC-005
    test('should show error when password is empty', async ({ page }) => {
        await loginPage.login(users.standard.username,'');
        const error = await loginPage.getErrorMessage();
        expect(error).toContain('Password is required');
    });

    // TC-006
    test('should show error when both fields are empty', async ({ page }) => {
        await loginPage.login('','');
        const error = await loginPage.getErrorMessage();
        expect(error).toContain('Username is required');
    });

    // TC-007
    test('should logout successfully', async ({ page }) => {
        await loginPage.login(users.standard.username, users.standard.password);
        await loginPage.logout();
        expect(await loginPage.loginButton.isVisible()).toBe(true);
    });
});
```

- test.beforeEach – navigates to the login page fresh before every single test case, so tests never depend on each other's state
- Each test calls loginPage.login(...) with different fixture data, then asserts either the resulting page title or the displayed error message

Step 11: Write the End-to-End Test (tests/e2e/checkout.spec.js)

Once login was verified, the next test written was the full happy-path purchase journey, chaining together every Page Object built so far: login → add to cart → go to cart → checkout → fill information → continue → finish → confirm.

```
const { test, expect } = require('@playwright/test');
const LoginPage     = require('../../pages/LoginPage');
const ProductsPage  = require('../../pages/ProductsPage');
const CartPage      = require('../../pages/CartPage');
```

```
const CheckoutPage  = require('../../pages/CheckoutPage');
const users         = require('../../fixtures/users');

test.describe('Checkout E2E', () => {
    test('should complete full purchase journey', async ({ page }) => {
        // Step 1: Login
        const loginPage = new LoginPage(page);
        await loginPage.navigate();
        await loginPage.login(users.standard.username, users.standard.password);

        // Step 2: Add product to cart
        const productsPage = new ProductsPage(page);
        await productsPage.addFirstItemToCart();

        // Step 3: Go to cart
        await productsPage.goToCart();

        // Step 4: Click Checkout
        const cartPage = new CartPage(page);
        await cartPage.clickCheckout();

        // Step 5: Fill info
        const checkoutPage = new CheckoutPage(page);
        await checkoutPage.fillInfo('Rama','Ahmed','1234');

        // Step 6: Click Continue
        await checkoutPage.clickContinue();

        // Step 7: Click Finish
        await checkoutPage.clickFinish();

        // Assert: Check confirmation message
        const msg = await checkoutPage.getConfirmation();
        expect(msg).toContain('Thank you for your order!');
    });
});
```

- This single test exercises all five Page Objects together in sequence, which is exactly what defines it as an End-to-End test rather than a unit-level check
- The final assertion confirms the exact confirmation text “Thank you for your order!” is displayed

Step 12: Write the Regression / Negative Test (tests/regression/negative.spec.js)

The last test written reuses the same purchase journey as Step 11, but deliberately submits the checkout form with an empty First Name field, to confirm the application's required-field validation keeps working correctly after any future code change.

```
const { test, expect } = require('@playwright/test');
const LoginPage     = require('../../pages/LoginPage');
const ProductsPage  = require('../../pages/ProductsPage');
const CartPage      = require('../../pages/CartPage');
const CheckoutPage  = require('../../pages/CheckoutPage');
const users         = require('../../fixtures/users');

test.describe('Negative test', () => {
```

```
    test('should complete full purchase journey', async ({ page }) => {
        // Step 1: Login
        const loginPage = new LoginPage(page);
        await loginPage.navigate();
        await loginPage.login(users.standard.username, users.standard.password);

        // Step 2: Add product to cart
        const productsPage = new ProductsPage(page);
        await productsPage.addFirstItemToCart();

        // Step 3: Go to cart
        await productsPage.goToCart();

        // Step 4: Click Checkout
        const cartPage = new CartPage(page);
        await cartPage.clickCheckout();

        // Step 5: Fill info (First Name left empty on purpose)
        const checkoutPage = new CheckoutPage(page);
        await checkoutPage.fillInfo('','Ahmed','1234');

        // Step 6: Click Continue
        await checkoutPage.clickContinue();

        // Assert: Check error message
        const error = await checkoutPage.getErrorMessage();
        expect(error).toContain('First Name is required');
    });
});
```

- Only one line differs from the E2E test: fillInfo('','Ahmed','1234') leaves the first name blank on purpose
- The assertion checks for the error message “First Name is required” instead of a confirmation message

Step 13: Run the Project & Review Results

With every file in place, the project is run and verified as follows.

Step 1: Open a terminal inside the project folder (the folder containing playwright.config.js).

```
cd path/to/playwright-ecommerce-tests
```

Step 2: Install dependencies and browsers (if not already installed).

```
npm install
npx playwright install
```

Step 3: Run the suites, starting with the fastest check first.

```
npm run test:smoke        # login tests only (Chromium)
npm run test:e2e          # full purchase journey only (Chromium)
npm run test:regression   # negative / validation test only
npm test                  # everything, on all 4 browser/device profiles
```

Step 4: Open the HTML report to review results, including screenshots/video/trace for any failures.

```
npm run report
```

- Smoke suite: all 7 login test cases passed, covering successful login, a locked-out user, invalid credentials, empty-field validation, and logout
- E2E suite: the full purchase journey completed successfully, ending in the expected order-confirmation message
- Regression suite: the checkout form correctly blocked submission and displayed the expected validation error when First Name was left empty
- Cross-browser run confirmed consistent behavior across Chromium, Firefox, WebKit, and the Pixel 5 mobile profile
- The GitHub Actions pipeline ran the same suite automatically and published the HTML report as a build artifact

[Insert screenshot here – Test Execution / HTML Report]

3. Conclusion

Following this walkthrough from configuration to execution reproduces the entire playwright-ecommerce-tests project: a Page Object Model framework covering login, product browsing, cart, and checkout, validated through smoke, end-to-end, and regression suites, executed across four browser/device profiles, and wired into an automated CI/CD pipeline. Anyone following these thirteen steps in order will end up with the exact same working project.